Duo: A Reciprocal World Model for Full-Duplex Interaction

Hengyuan Cao, Biao Gong†, Guoshan Lu, Qiyuan Zhang, Lin Liu, Yiding Tian, Yilei Jiang, Yuxin Ma, Chengzhang Wu, Kun Chen, Yiqian Xie, Ting Liu, Xin Zheng, Xijun Gu, Tianren Ma, Jiancheng Pan, Chi Zhang, Xuzhe Zheng, Siyi Li, Zhenzhong Lan†

†Project Lead

arXiv:2611.15131v1 [cs.CV] 11 Nov 2026

Abstract

In this report, we introduce the Reciprocal World Model (RWM) and present **Duo v1**, a full-duplex world model that continuously perceives audiovisual inputs while responding to users in real time. The RWM treats intelligence and physical reality as equally fundamental to what constitutes a world. Existing world models largely focus on simulating the latter, leaving reasoning and decision-making as external controls rather than intrinsic parts of the world itself. In RWM, we identify two equally fundamental capabilities: *Intent*, which provides the cognition underlying world interactions, and *Realization*, which generates coherent audiovisual behavior. To realize this formulation, **Duo v1** introduces three key technical designs: (1) Leveraging autoregressive modeling for continuous interaction reasoning and participation decisions, rather than restricting it to instruction understanding for diffusion models. (2) Constructing chunk-specific conditions and synchronize tokens across Intent and Realization on a unified timeline. Module-specific attention masks enforce causal dependencies both within and across chunks, preventing access to future inputs. (3) Mitigating distribution shift in both modules through dual rollout distillation, training on autoregressively generated Intent tokens and synthesized audiovisual histories from Realization. Both modules continuously perceive real-time audio, video, and text, enabling full-duplex interaction with spontaneous conversations and interruptions. Beyond conversation, users can share real objects, reshape virtual scenes, guide assistants, or play audiovisual games without mouse or keyboard controls. The release of **Duo v1** makes our first step, with the next version coming soon. We look forward to broader exploration of this emerging field.

Homepage https://duo-v1.github.io

GitHub https://github.com/inclusionAI/Duo

Hugging Face Model https://huggingface.co/collections/inclusionAI/duo-v1



Figure 1: Illustrative scenarios for full-duplex interaction with Reciprocal World Model.

Contents

1 Introduction **3**

2 Related Work **3**

3 Architecture **4**

3.1 The Overview of Reciprocal World Model 4

3.2 Multimodal Intent Module 6

3.3 Variational Autoencoder 6

3.4 Joint Causal Audiovisual Realization 6

4 Training Strategy **8**

4.1 Stage 1: Intent Training 8

4.2 Stage 2: Realization Training 9

4.3 Stage 3: Distribution Matching Distillation 10

5 Data Construction **13**

5.1 Synthetic Data 13

5.2 Recorded Data 14

5.3 Data Processing 16

5.3.1 Synthetic Data Quality Control 16

5.3.2 Live-Stream Data Processing 16

6 Streaming Pipeline and Acceleration **18**

6.1 Orchestrated Streaming Pipeline 18

6.2 Causal Latent Decoding 18

6.3 Inference Acceleration 18

7 Evaluation and Showcase **20**

7.1 Human Evaluation 20

7.2 Showcase 20

8 Conclusion **21**

1 Introduction

World models have made rapid progress in simulating the physical side of a world (Ha & Schmidhuber, 2018; Hafner et al., 2025; Brooks et al., 2024): given past observations and control signals, they predict how scenes evolve (Bruce et al., 2024; Valevski et al., 2025; Hu et al., 2023; NVIDIA, 2025), with increasingly realistic dynamics, appearance, and long-range consistency (Google DeepMind, 2024; Parker-Holder & Fruchter, 2025; Robbyant Team, 2026; Gao et al., 2026; Chen et al., 2026; Huang et al., 2026a). In most of these works, however, intelligence resides outside the world. A user steers the generated world through inputs with limited degrees of freedom, such as keyboard and mouse commands, gamepad actions, or text prompts, but nothing within the world perceives the full state of the user in real time or decides how to respond. Such worlds can be accurate, yet they remain lifeless: *there is a world to explore, but no one to talk to.*

We argue that intelligence and physical reality are equally fundamental to a world. Building on this view, we introduce the `Reciprocal World Model` (`RWM`). The `RWM` generates a world inhabited by living presences and connects it to the real world through full-duplex, real-time interaction. Each side continuously perceives audio and video from the other and can respond, interrupt, or remain silent while the other is still acting. This reciprocal interaction requires two complementary capabilities: *Intent*, which provides the cognition underlying world interactions, and *Realization*, which generates coherent audiovisual behavior. These functional roles can be fulfilled by a unified multimodal model or by a modular system combining a multimodal language model with a diffusion model. In this report, we present **`Duo v1`**, a modular instantiation of the `RWM`. Interaction proceeds in chunks of 17 video frames, approximately 0.71 s at 24 fps. Three technical designs coordinate Intent and Realization within a single streaming system.

Autoregressive intent for interaction. In most audio-visual video generative model (MiniMax, 2026), a language model serves as an instruction encoder for the diffusion model. In **`Duo v1`**, the intent module instead reasons over the ongoing interaction autoregressively and makes participation decisions: at every chunk, it emits a `<speak>` action followed by text or a `<wait>` action, conditioned on the user input so far and its own earlier language. Its hidden states, rather than its decoded text alone, carry each decision to the realization module (Sec. 3). Action-aware rebalancing prevents the intent module from collapsing into perpetual waiting, since most chunks contain no new assistant speech, and history perturbations teach it to recover from imperfect earlier responses (Sec. 4).

Causal chunk conditions on a unified timeline. User speech, user video, intent tokens, and assistant media are synchronized on one timeline, and each chunk is given exactly the conditions that are available at its decision boundary. Module-specific attention masks enforce causality within and across chunks. In the intent module, temporal patches are completed inside each chunk, so no future frame leaks into a decision through the visual frontend. In the realization module, noisy media tokens of a chunk can attend to the reference, intent features, and clean history, but not to their own clean targets or to any later chunk (Sec. 3).

Dual rollout distillation. Before distillation, the realization module is trained on intent features of recorded assistant text and on recorded media history, whereas in a live exchange both are produced by the system itself. Dual rollout brings inference-time conditioning into distillation: within each training window, the frozen Intent Module autoregressively generates the final assistant turn, and the Realization Module conditions on the resulting intent features and its own generated audiovisual history. Distribution matching distillation on these rollouts yields a four-step streaming generator trained under conditions that match live interaction.

We develop several data construction pipelines and a streaming runtime to support **`Duo v1`**. The training data combine scripted synthetic video calls and recorded conversations, with audio, video, and text aligned on a shared timeline for causal interaction training (Sec. 5). The streaming pipeline coordinates intent inference, audiovisual generation, and incremental decoding while continuing to capture user inputs (Sec. 6). Speech and vision provide a rich, direct channel between the two worlds, conveying information that keyboard and mouse commands often fail to capture, including tone, timing, expression, and visual details of real objects, actions, and scenes. This richer interface creates opportunities for a wide range of shared activities shaped jointly by real and generated participants.

2 Related Work

World models and persistent interaction. World Models and Dreamer learn predictive representations for control (Ha & Schmidhuber, 2018; Hafner et al., 2025), while Genie learns action-controllable environments from video (Bruce et al., 2024). LingBot-World studies a data engine and staged causal adaptation for

continuing visual worlds (Robbyant Team, 2026). These works connect learned dynamics to an agent's actions and subsequent observations.

Full-duplex and multimodal participation. Moshi represents concurrent user and assistant speech streams (Défossez et al., 2024). Qwen-Omni separates multimodal understanding from streaming speech generation through connected Thinker and Talker components (Xu et al., 2025a;b). Wan-Streamer extends the streaming setting to audio–visual foundation models and their execution schedule (Huang et al., 2026b). ELLSA jointly models full-duplex speech, gesture, and expression, including non-verbal behavior while listening (Wang et al., 2026). These works establish participation timing and visible communicative behavior as closely related modeling problems.

Joint and streaming audiovisual generation. Ovi, MOVA, and LTX develop synchronized audio–visual generation through coupled modality streams (Low et al., 2025; SII-OpenMOSS Team, 2026; HaCohen et al., 2026). MiniMax-H3 is a reference-conditioned audio–visual generator whose weights can initialize related models (MiniMax, 2026). Audio-driven animation methods such as EMO and OmniHuman realize visible behavior from supplied speech (Tian et al., 2024; Lin et al., 2025). LiveTalk combines a distilled video performer with streaming audio from Qwen-Omni (Chern et al., 2025). These approaches connect speech with visible articulation and motion, with different choices of conditioning and generation order. Streaming joint diffusion further requires audio and video to remain aligned as their generated histories grow. OmniForcing addresses this conversion through asymmetric block-causal alignment, audio-sink stabilization, and joint self-generated rollouts (Su et al., 2026). OmniLiveAvatar combines progressive autoregressive distillation, synchronized audio-video memory, and rolling prompt planning (Zhu et al., 2026b).

Causal adaptation and few-step training. Diffusion Forcing assigns different noise levels across a sequence, and CausVid distills a bidirectional generator into a causal few-step student (Chen et al., 2024; Yin et al., 2025). Self Forcing uses self-generated history during training to reduce the context mismatch with autoregressive inference (Huang et al., 2025). Work on causal-compatible initialization further aligns teacher and student during the transition to few-step generation (Zhu et al., 2026a; Zhao et al., 2026). Consistency models and distribution matching provide complementary tools for reducing sampling steps (Song et al., 2023; Yin et al., 2024b), with later distribution-matching work incorporating multistep alignment and adversarial training (Yin et al., 2024a).

3 Architecture

Duo v1 couples a language-based *Intent Module* with a causal audiovisual *Realization Module* (Fig. 2). Hidden states produced by the Intent Module serve as the interface between the two modules, providing semantic guidance for the joint generation of video and stereo audio for the assistant. The two modules process complementary inputs and maintain separate histories, integrating response planning and audiovisual synthesis within a unified architecture. The following subsections describe this design in detail: Sec. 3.1 formalizes the interaction between the two modules, Sec. 3.2 details the Intent Module, Sec. 3.3 describes the audio-visual latent space, and Sec. 3.4 presents the causal joint generator used by the Realization Module.

3.1 The Overview of Reciprocal World Model

RWM treats intelligence and physical reality as equally fundamental components of a world. Their interaction is reciprocal: Intent determines how the system responds to the evolving environment, while Realization expresses that intent through audiovisual behavior, which in turn becomes part of the environment for subsequent interaction. We formalize this process as a chunk-wise, full-duplex interaction.

An interaction is divided into consecutive chunks of fixed duration, and $\mathbf{c}_k$ denotes the chunk at time index k. A chunk is a time interval and need not align with a word, a sentence, or a conversational turn. For each chunk $\mathbf{c}_k$, let u_k^v, u_k^t, and u_k^a denote the incoming video, speech transcript, and audio, respectively. Let y_k denote the language output generated by **Duo v1**, either <speak> followed by text or <wait>. Let h_k denote the hidden states produced by the Intent Module for this output, and $z_k = (z_k^v, z_k^a)$ the latent representations of the generated video and audio. We use the subscript $< k$ to denote the history of a sequence up to, but not including, chunk $\mathbf{c}_k$. d denotes the global instructions, and I_{ref} denotes the reference image.

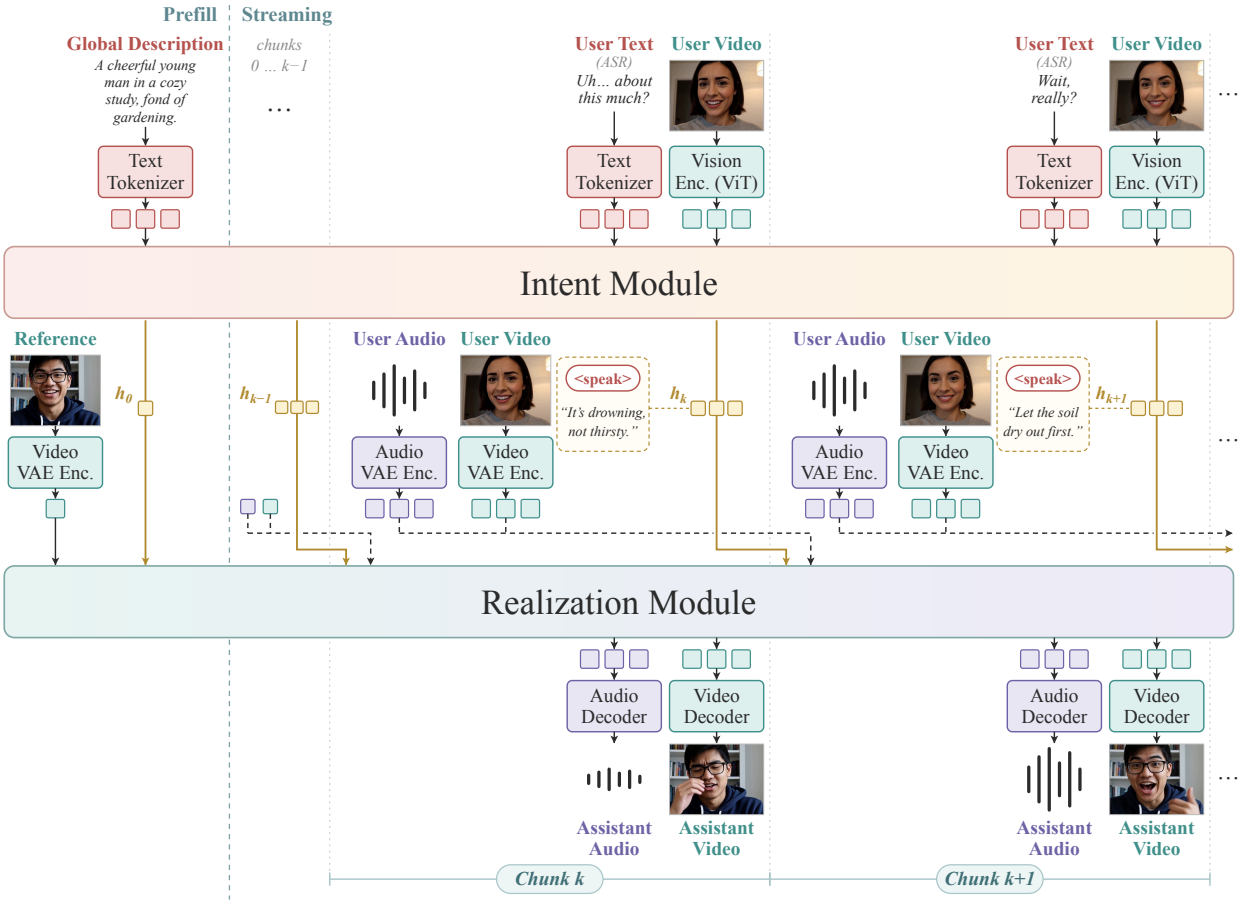


Figure 2: **The framework of Duo v1.** A joint causal diffusion transformer is conditioned on hidden states from the Intent Module, directly encoded user video and audio, a visual reference, and previously generated assistant media. The conversation media illustrate an offline, script-conditioned example.

At each chunk k, **Duo v1** generates a response $\mathbf{c}_k$ in two steps, where $< k$ denotes the history before chunk k. First, the Intent Module conditions on the global instructions d, user video embeddings $x^v_{u,<k}$, user text $u_{t,<k}$, and previous assistant text outputs $y_{<k}$:

$$C^\phi_k = (d,\ x^v_{u,<k},\ u_{t,<k},\ y_{<k}),$$
$$y_k \sim p_\phi(\cdot \mid C^\phi_k), \qquad h_k = f_\phi(C^\phi_k,\ y_k). \tag{1}$$

The model forward pass produces the text output y_k and hidden states h_k, which convey the intent to the Realization Module. The Realization Module jointly generates audio and video latents, conditioned on the reference image I_{ref}, the intent hidden states h_k, user video embeddings $x^v_{u,<k}$, user audio $u_{a,<k}$, and previously generated assistant latents $z^a_{<k}$ and $z^v_{<k}$:

$$C^\theta_k = (I_{\text{ref}},\ h_k,\ x^v_{u,<k},\ u_{a,<k},\ z^a_{<k},\ z^v_{<k}),$$
$$(z^a_k,\ z^v_k) \sim p_\theta(\cdot \mid C^\theta_k). \tag{2}$$

The generated latents z^a_k and z^v_k are decoded into the audio and video of the response chunk $\mathbf{c}_k$.

This formulation has two properties. First, it is causal. Both modules condition on interaction histories preceding chunk k, and the Realization Module additionally receives the current intent representation h_k. Neither module accesses future inputs. User inputs received during response generation can be incorporated into subsequent chunks, supporting full-duplex interaction. Second, it supports continuity across chunks. The text output y_k and generated latents z^a_k and z^v_k are added to the corresponding histories for subsequent

chunks, allowing the system to continue from previous text, audio, and video outputs. Sec. 3.2 describes how the Intent Module implements p_ϕ, Sec. 3.3 describes the audio-visual latent representations z_k^a and z_k^v, and Sec. 3.4 describes how the Realization Module implements p_θ.

3.2 Multimodal Intent Module

The Intent Module adapts a pretrained multimodal language backbone (Bai et al., 2025) to causal, chunk-wise interaction. Given the context, it generates the text output y_k and provides hidden states h_k.

Causal input assembly. Inputs enter C_k^ϕ according to when they become available, without requiring all modalities to share a sampling rate. ASR recovers user text u_t from user audio u_a, and forced alignment assigns each text token a span on the source audio timeline. A token is admitted only after the end of its aligned span. Video frames and audio samples are mapped to the same half-open chunk intervals using source timestamps.

Assistant text targets are assigned to chunks according to speech onset, while the initial chunk uses a fixed <wait> action (Fig. 3). We preserve fine-grained alignment before constructing chunk views, avoiding early rounding of speech boundaries. Each input context therefore contains only information available at the corresponding decision boundary, and different chunk configurations can be derived from the same aligned record. The ASR and alignment tools, together with their quality checks, are described in Sec. 5.3.

Causal visual encoding. Causality must also hold before attention is applied. Temporal patch construction can combine multiple source frames, so an attention mask cannot remove future information already mixed into a visual token. We therefore form each temporal group entirely within one input chunk. For a native 17-frame chunk with source frames numbered $0, \ldots, 16$, the final pair is $(16, 16)$: the last frame is repeated instead of paired with frame 17 from the next chunk. Each group becomes available at the latest timestamp among its source frames. This preserves the pretrained visual input format while preventing temporal grouping from crossing chunk boundaries.

Participation and language outputs. The text output y_k contains either <speak> followed by a text body or <wait> without new text. Both actions are generated through the standard token prediction path. The explicit action separates the decision to begin speaking from the words of the response, since appropriate content can still be delivered at the wrong time. A <wait> output can coexist with ongoing playback and media continuation. Participation supervision, target balancing, and history augmentation are described in Sec. 4.1.

3.3 Variational Autoencoder

To support real-time response generation, we encode audio and video into compact latent representations z^a and z^v. For audio, each stereo channel is compressed into 40 latent vectors per second, with 32 dimensions per vector. For video, the encoder processes groups of 17 frames, with padding applied when fewer than 17 frames are available. Each group consists of one keyframe followed by four packets of four frames, following a 1-4-4-4-4 temporal arrangement. These 17 frames are encoded into 5 latent frames. Each latent frame has 24 dimensions, with spatial resolution reduced by a factor of 16×16. For input height H and width W, an RGB frame group of shape $(17, H, W, 3)$ is therefore encoded into a latent:

$$z^v_{\text{group}} \in \mathbb{R}^{5 \times \frac{H}{16} \times \frac{W}{16} \times 24}. \tag{3}$$

Before entering the diffusion transformer, the video latents undergo spatial merging over 2×2 patches, producing a video input vector with $2 \times 2 \times 24 = 96$ dimensions. Each audio latent vector has 32 dimensions per channel. The decoding procedure is described in Sec. 6.

3.4 Joint Causal Audiovisual Realization

The Realization Module uses a causal audiovisual diffusion transformer initialized with pretrained weights from (MiniMax, 2026). Causal adaptation and distillation of the pretrained backbone are described in Sec. 4.2. The module jointly generates audio and video in the latent spaces described in Sec. 3.3, implementing the conditional generation process in Eq. (2) through flow matching (Lipman et al., 2023). Starting from random noise, the model generates audiovisual latents conditioned on the intent representation, reference image, available user inputs, and previously generated assistant media. Access to this history supports continuity across response chunks.

Intent conditioning. A linear projection maps h_k from the Intent Module to the hidden width of the Realization Module, and a refiner network produces text-conditioning embeddings. An attention mask

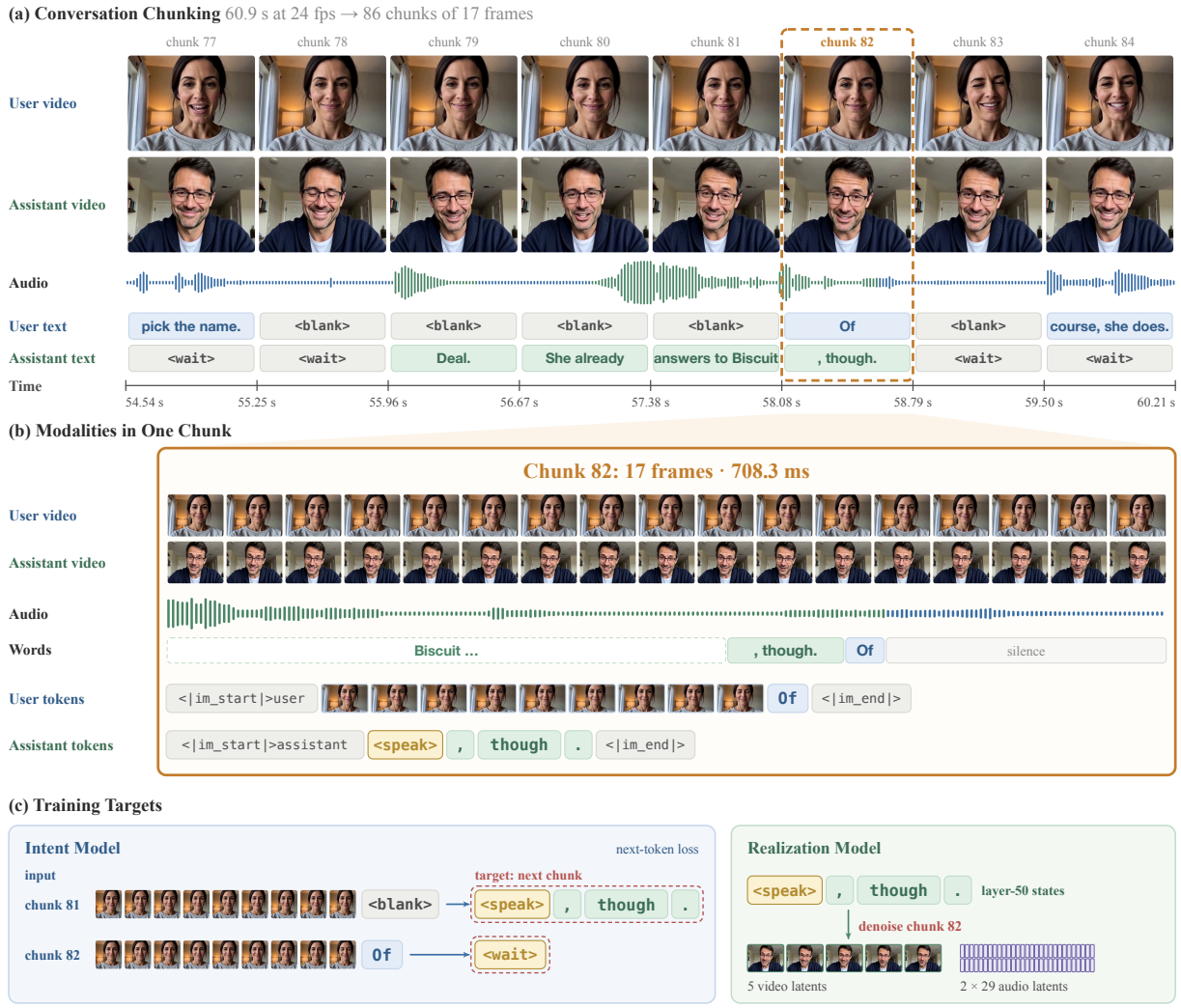


Figure 3: **ASR-aligned chunk construction and causal supervision.** On a shared audiovisual timeline, user text is assigned to chunks by aligned token end times, while assistant text targets are assigned by speech onset. The token <blank> marks the absence of newly available user text. An assistant chunk with no new speech onset uses <wait>, even if speech from the preceding chunk continues. At chunk k, the Intent Module predicts y_k from C_k^{ϕ}, using user inputs available through chunk $k-1$. The initial output y_0 is fixed to <wait>. Only subsequent assistant targets contribute to the loss.

restricts feature computation to token positions associated with the global instructions d and the current assistant text output y_k:

$$x_k^t = \mathrm{Refine}_\theta\left(\mathrm{Project}_\theta(h_k); M_k^{\mathrm{feat}}\right). \tag{4}$$

Media conditioning. User audio and video are encoded by the respective audio and video VAE encoders described in Sec. 3.3, producing conditioning latents $z^a_{u,<k}$ and $z^v_{u,<k}$. The reference image I_{ref} is encoded by the video VAE into reference latents z^r. These latents are mapped to the corresponding input embeddings $x^a_{u,<k}$, $x^v_{u,<k}$, and x^r for the joint DiT. Text conditioning is provided by x^t_k, derived from hidden states associated with the global instructions d and the current assistant output y_k, as described above. Previously generated assistant latents $z^a_{<k}$ and $z^v_{<k}$ provide audiovisual history. Together, these conditions supply user observations, reference appearance, semantic guidance, and generated history for the current response chunk. Only user inputs available before chunk k are included.

Positional embedding. We use 3D-RoPE to represent temporal and spatial relationships among tokens. The text-conditioning embeddings use only the temporal coordinate, with both spatial coordinates set to zero. Audio tokens use the two extreme spatial position IDs to distinguish the left and right channels.

Temporal coordinates for audio and video are aligned with actual playback times. At 40 latent vectors per second per channel, each audio latent corresponds to 25 milliseconds. The beginning of the user input audio defines the common temporal origin. Each audio token receives a temporal position equal to the offset of its segment start from this origin, divided by 25 milliseconds. Video temporal positions use the corresponding playback times with the same origin and temporal unit. This accounts for the nonuniform temporal mapping from 17 source frames to five video latent frames.

4 Training Strategy

Training proceeds in three stages. Stage 1 trains the Intent Module for causal participation and text prediction through supervised fine-tuning. Stage 2 trains the Realization Module for causal joint audiovisual generation, conditioned on recorded media history and intent features extracted from recorded assistant text by the frozen Intent Module (Fig. 4). Stage 3 distills four-step generation through distribution matching, with dual rollout using predicted text and generated audiovisual history to match inference-time conditioning.

4.1 Stage 1: Intent Training

We adapt the multimodal Intent Module with causal supervised fine-tuning on chunk-aligned conversations. At the start of chunk k, the model predicts the assistant output text y_k from the global instruction d, the user video u_v and ASR text u_t available through chunk $k-1$, and earlier assistant language $y_{<k}$. The decision context excludes assistant video. In the training data, user tokens become available at the ends of their aligned spans, while assistant text tokens are assigned to the chunk containing their speech onset. The <speak> action introduces new text, including continuations of an earlier turn. The <wait> action adds no new text and can coexist with ongoing playback or non-verbal behavior. Fig. 3 illustrates this shift, and Sec. 3.2 details the input protocol.

Causal chunk supervision. Let y_k be the token sequence of the assistant output in chunk k, and let $C_k^{\mathrm{I}} = (d, u_{v,<k}, u_{t,<k}, y_{<k})$ denote the causal context of the Intent Module. We minimize the weighted autoregressive objective

$$\mathcal{L}_{\text{intent}} = -\mathbb{E}\left[\frac{\sum_{k,j} m_{k,j} w_{k,j} \log p_\phi\left(y_{k,j} \mid C_k^{\mathrm{I}}, y_{k,<j}\right)}{\max\left(1, \sum_{k,j} m_{k,j} w_{k,j}\right)}\right], \tag{5}$$

where $y_{k,j}$ is the j-th action, content, or structural token, $w_{k,j}$ is its category-dependent weight, and $m_{k,j}$ is one for supervised tokens and zero otherwise. The denominator sums the effective weights within each sampled conversation, with a minimum of one. The initial assistant output y_0 is a fixed <wait> used as context and excluded from the loss.

Action-aware rebalancing. Most chunks introduce no new assistant text, so unweighted training can favor waiting. We assign supervision categories offline from recorded assistant events and from whether the user is speaking at the chunk start. This speaking status distinguishes idle waits from waits during user speech and ordinary openings from interruption openings. The assistant annotation distinguishes openings from continuations, and an annotated question in the preceding user turn provides a separate opening tag. We use these labels to subsample targets and weight the loss, with separate action and content weights and adjustments for opening categories. Subsampled targets remain in the conversation history. The model receives only the causal context described above. Validation retains the full set of eligible targets and reports separate token-level metrics for actions, speech, and waits.

Training under imperfect assistant history. To expose the model to imperfect histories, we truncate or repeat text in selected continuation chunks, mask their losses, and retain supervision on later valid targets. For missed-opening recovery, we replace selected ordinary openings with <wait> in the history and mask their losses. In the next chunk of the same assistant turn, only <speak> contributes to the loss.

Preserving visual and linguistic grounding. The training mixture contains examples whose correct answer depends on the user video as well as examples solvable from text alone. We identify visually grounded assistant turns through data inspection and upweight their content-token loss, while retaining the same causal visual input path. Prompt augmentation varies the format, ordering, and completeness of the static

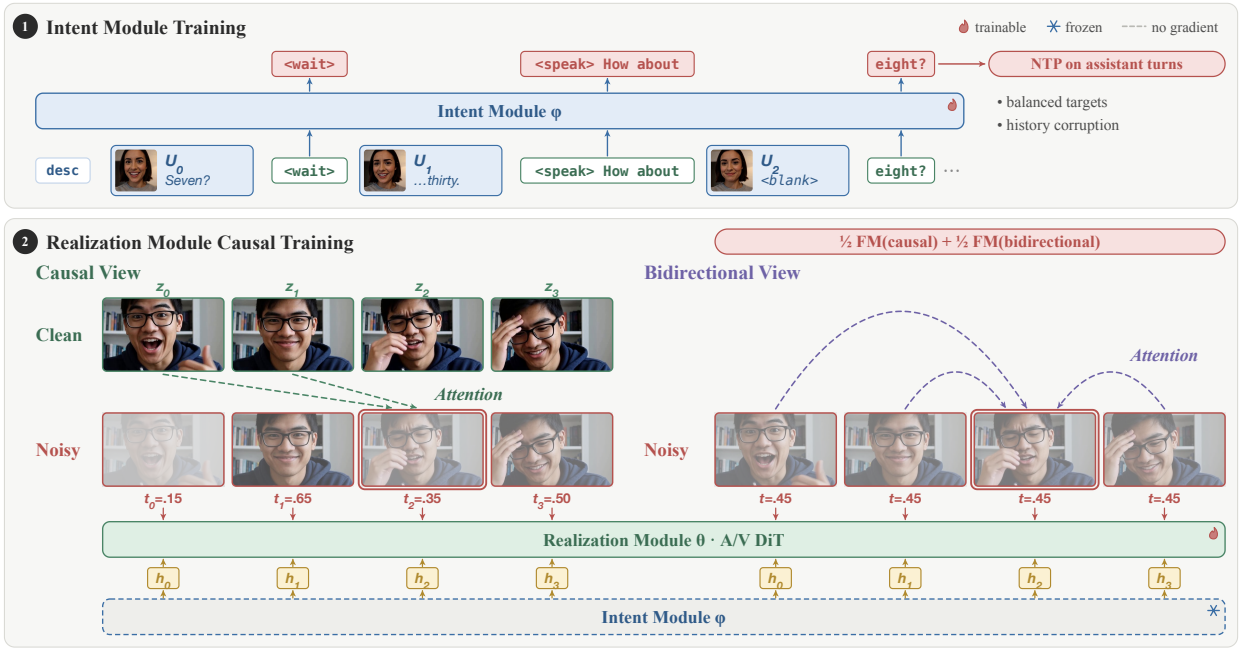


Figure 4: Overview of Stage 1 (Intent Training) and Stage 2 (Realization Training).

assistant/scene description without inventing new facts. Together with source-grouped train/validation splits, these choices discourage the Intent Module from relying on a fixed prompt template or on text-only shortcuts. Optional emotion labels are kept as separately weighted attachments and are admitted only at their annotated causal boundary. We use low-rank adaptation to fine-tune the Intent Module. After this stage, the frozen Intent Module supplies hidden states from its forward passes to condition media training.

4.2 Stage 2: Realization Training

Stage 2 trains the Realization Module for causal joint audiovisual generation while keeping the Intent Module frozen. Training follows the conditional structure in Eq. (2), learning to generate audio and video latents from the current intent representation and available interaction context. The sequence layout and attention mask encode the dependencies among modalities within each chunk and restrict access to preceding interaction history across chunks. We describe this multimodal sequence construction, the joint flow-matching formulation, and the optimization strategy below.

Chunk structure and sequence design. Each segment of streaming generation spans 17 video frames at 24 fps, or $17/24 \approx 0.708$ seconds. The chunk $\mathbf{c}_k$ contains the assistant action for that chunk, represented by the text embedding x_k^t obtained from the Intent Module hidden state h_k, together with the corresponding noised assistant video and audio embeddings:

$$\mathbf{c}_k = (x_k^t, \tilde{x}_k^v, \tilde{x}_k^a), \tag{6}$$

where $\tilde{x}_k^v$ and $\tilde{x}_k^a$ are the clean media embeddings x_k^v and x_k^a noised to level τ_k. To align with the timelines used by the audio and video codecs, the number of media embeddings in each chunk corresponds to 5 latent video representations and 2×29 latent audio representations.

During training, we also insert clean audio and video embeddings into the sequence. A carefully designed attention mask prevents noisy media embeddings in each generation chunk from accessing their own clean signals. For each training example, the sequence contains, in order, the assistant reference-media embeddings, the global instruction embeddings, the clean media embeddings for the generation chunks, and the generation chunks themselves:

$$C^{\mathrm{R}} = (x^r, x^d, x_0^v, x_0^a, \ldots, x_K^v, x_K^a, \mathbf{c}_0, \ldots, \mathbf{c}_K), \tag{7}$$

where x^r embeds the reference image I_{ref}, x^d embeds the global instruction d, and $K+1$ is the number of chunks in the sample.

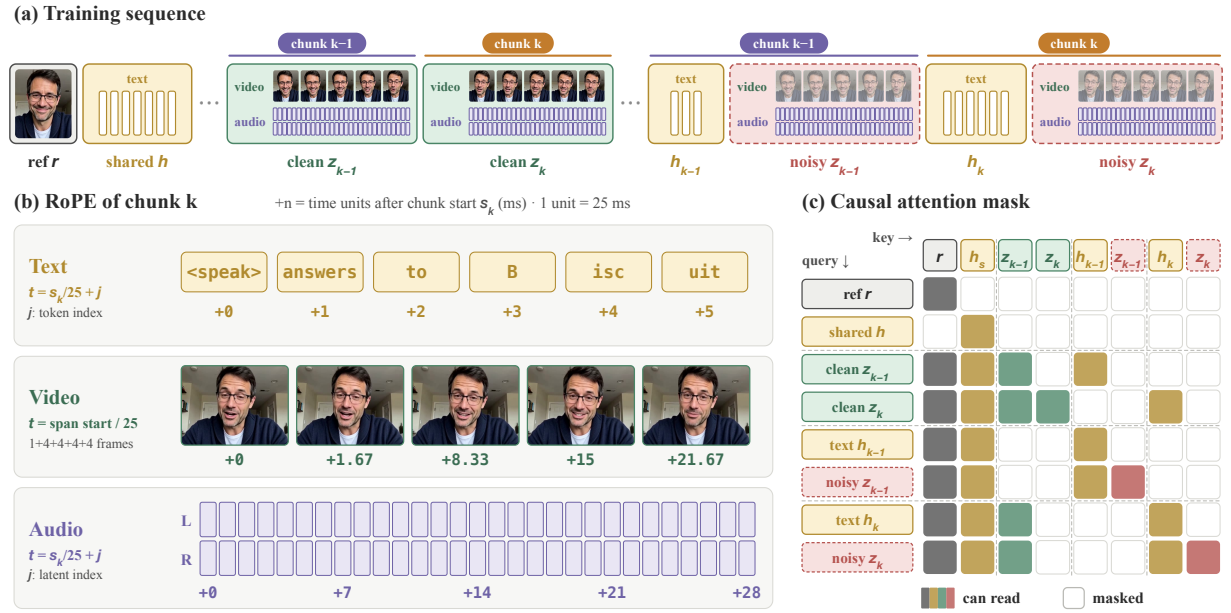


Figure 5: **Training sequence, positional encoding, and attention mask of the Realization Module.** (a) The training context C^{R} contains the reference, the shared global instruction, clean media of all chunks, and, for each chunk, the Intent Module hidden states followed by the noisy media. (b) Temporal RoPE positions in chunk k follow playback time in 25 ms units from the chunk start s_k: text tokens and audio latents advance by one unit per index, and each video latent takes the start time of its frame span. (c) Causal attention mask. The noisy media of chunk k read the reference, the shared instruction, clean media of earlier chunks, the hidden states of chunk k, and themselves, but not the clean media of chunk k or any later chunk.

To fit within the context length of the initialized model, we randomly sample a window of consecutive chunks from each training conversation. Because temporal position IDs align with actual video playback times, positional encoding remains closely aligned between training and inference even when the sampled chunk sequence does not start at the beginning of a conversation.

Attention mask design. Each noisy media embedding can attend to the reference media, global description, historical assistant text, and clean historical media. Clean media related to the current chunk and all content in future chunks are invisible. Noisy media embeddings within the current chunk also have internal content boundaries: video embeddings correspond in order to 5 video-encoder latent representations, while audio embeddings are divided into left and right channels, each corresponding to 25 milliseconds of audio. Nevertheless, all these embeddings are mutually visible within the chunk. The detailed visibility relationships are illustrated in Fig. 5.

Flow-matching velocity prediction. Flow matching (Lipman et al., 2023) supervises the joint media velocity from random noise to clean latent states, with clean time $\tau = 1$ and noise time $\tau = 0$. For clean media latents $z \in \{z^v, z^a\}$ and Gaussian noise ϵ, we have:

$$\begin{aligned} \tilde{z}_\tau &= (1-\tau)\epsilon + \tau z \qquad \text{noised input,} \\ v^\star &= z - \epsilon \qquad\qquad\quad \text{target velocity.} \end{aligned} \tag{8}$$

4.3 Stage 3: Distribution Matching Distillation

DMD supervision. DMD (Yin et al., 2024b) distills the causal model into a four-step chunk-causal generator G_θ by matching the distribution of its own samples to the data distribution, instead of enforcing agreement between neighboring points on a teacher trajectory as CD does. Two score models provide this supervision: a frozen real-score model μ_{real} approximates the conditional data distribution, and a trainable fake-score model μ^{ψ}_{fake} tracks the evolving generator distribution. The generator synthesizes a latent window z_{gen} causally, whereas both score models evaluate the completed window bidirectionally. In our flow convention ($\tau = 0$ noise, $\tau = 1$ data), each conversation draws one noise level τ from a shifted discrete noise schedule,

the window is corrupted once as $\tilde{z}_{\text{gen}} = \tau z_{\text{gen}} + (1-\tau)\epsilon$, and the two models return clean estimates $\hat{z}_{\text{real}} = \mu_{\text{real}}(\tilde{z}_{\text{gen}}, \tau)$ and $\hat{z}_{\text{fake}} = \mu^{\psi}_{\text{fake}}(\tilde{z}_{\text{gen}}, \tau)$ under this shared corruption. Since a clean estimate $\hat{z}_*$ determines the score $s_* = -(\tilde{z}_{\text{gen}} - \tau\hat{z}_*)/(1-\tau)^2$, the DMD gradient of the reverse KL divergence reduces to the difference of the two estimates,

$$\nabla_\theta D_{\text{KL}}(p_\theta \,\|\, p_{\text{data}}) \approx \mathbb{E}_{\tau,\epsilon}\left[(s_{\text{fake}} - s_{\text{real}})^\top \frac{\partial z_{\text{gen}}}{\partial \theta}\right], \qquad s_{\text{fake}} - s_{\text{real}} = \frac{\tau}{(1-\tau)^2}\,(\hat{z}_{\text{fake}} - \hat{z}_{\text{real}}). \tag{9}$$

We replace the time-dependent factor by a sample-adaptive scale and realize the update as regression toward a detached target; for one conversation and one modality,

$$\kappa = \max\big(\langle |z_{\text{gen}} - \hat{z}_{\text{real}}| \rangle_w,\ \varepsilon_0\big), \qquad \Delta = \frac{\hat{z}_{\text{fake}} - \hat{z}_{\text{real}}}{\kappa}, \qquad z_{\text{DMD}} = \text{sg}\big[z_{\text{gen}} - \Delta\big], \tag{10}$$

where sg denotes stop-gradient, $\langle\cdot\rangle_w$ averages over feature dimensions and then over the valid target rows of that conversation and modality, and the floor ε_0 prevents division by a vanishing scale. Assembling the targets across conversations yields the generator loss

$$\mathcal{L}_{\text{DMD}} = \frac{1}{2} \sum_{m \in \{v,a\}} \lambda_m \,\text{WMSE}_{w_m}\Big(z^m_{\text{gen}}, z^m_{\text{DMD}}\Big), \tag{11}$$

whose WMSE reduction uses the global valid-target normalization of Sec. 4.2. Its gradient with respect to z_{gen} is proportional to Δ, so the generator is moved along $(\hat{z}_{\text{real}} - \hat{z}_{\text{fake}})/\kappa$ through its differentiable outputs.

Because the fake model must follow the generator, it is fitted on detached generated samples by flow matching, $\mathcal{L}^{\text{fake}}_{\text{FM}} = \mathbb{E}\big[\omega(\tau)\,\|v_\psi(\tilde{z}_{\text{gen}}, \tau) - (z_{\text{gen}} - \epsilon)\|^2\big]$, where $\omega(\tau)$ is a noise-level-dependent weight. The fake model is updated more frequently than the generator, with several fake updates between consecutive generator updates. Each update draws its own window and rollout, so the fake model is always fitted to samples from the current generator.

An optional GAN loss (Yin et al., 2024a) adds direct feedback from training media. A lightweight head on middle-block features of the fake-score backbone pools the valid audio and video tokens of each conversation through a learned attention query into a single logit $D(\cdot)$. Its input physically excludes text, Intent Module, reference, and clean-context rows, so the critic judges joint audio–visual realism without requiring generated media to reproduce the recorded transcript. Writing $\tilde{z}_{\text{data}}$ and $\tilde{z}_{\text{gen}}$ for training-media and generated latents noised to the same level with independent noise, the two trainable roles minimize

$$\begin{aligned} \mathcal{L}_G &= \mathcal{L}_{\text{DMD}} + \lambda_G\,\mathbb{E}\big[\text{softplus}\big(-D(\tilde{z}_{\text{gen}})\big)\big], \\ \mathcal{L}_F &= \mathcal{L}^{\text{fake}}_{\text{FM}} + \lambda_D\,\mathbb{E}\big[\text{softplus}(-D(\tilde{z}_{\text{data}})) + \text{softplus}\big(D(\tilde{z}_{\text{gen}})\big)\big], \end{aligned} \tag{12}$$

where the fake backbone and head are frozen but differentiable in the generator update, and generated media are detached in the critic update.

Dual rollout in DMD training. Dual rollout aligns DMD training with the language and media context encountered during inference (Fig. 6). Before DMD, the Realization Module conditions on features extracted from layer 50 of the Intent Module using recorded assistant text, together with recorded media prefixes. At inference, the Intent Module determines response wording, length, and speaking timing, while the Realization Module continues from its own audiovisual outputs. Each DMD sample therefore undergoes a language rollout by the frozen Intent Module, followed by a media rollout by the Realization Module.

The frozen Intent Module rolls out only the final assistant turn of each training window. Windows may start at any earlier chunk and end at a natural assistant turn boundary rounded up to a codec chunk. Candidates are bucketed by reserved token length, with bucket weights proportional to squared length. Let k_0 be the first chunk within the window containing the final turn. Using only history within the window (Sec. 3.1), the Intent Module replays recorded text before k_0 and greedily predicts participation actions and text thereafter. Recorded tokens of the final turn and all later text are removed, while any recorded prefix from an earlier assistant turn sharing the boundary chunk is retained. Media rollout follows Self Forcing (Huang et al., 2025):

$$\begin{aligned} \widehat{h}_k &= \begin{cases} \text{Intent}_\phi\big(C^{\text{I}}_k;\, y_k\big), & k < k_0 \quad \text{(recorded history)}, \\ \text{Intent}_\phi\big(C^{\text{I}}_k\big), & k \geq k_0 \quad \text{(intent rollout)}, \end{cases} \\ \widehat{z}_k &= G^{(j^\star)}_\theta\Big(\epsilon_k;\, x^r, x^d, \widehat{h}_k, \text{sg}[\widehat{z}_{<k}]\Big), \qquad j^\star \sim \mathcal{U}\{0, \ldots, N-1\}, \end{aligned} \tag{13}$$

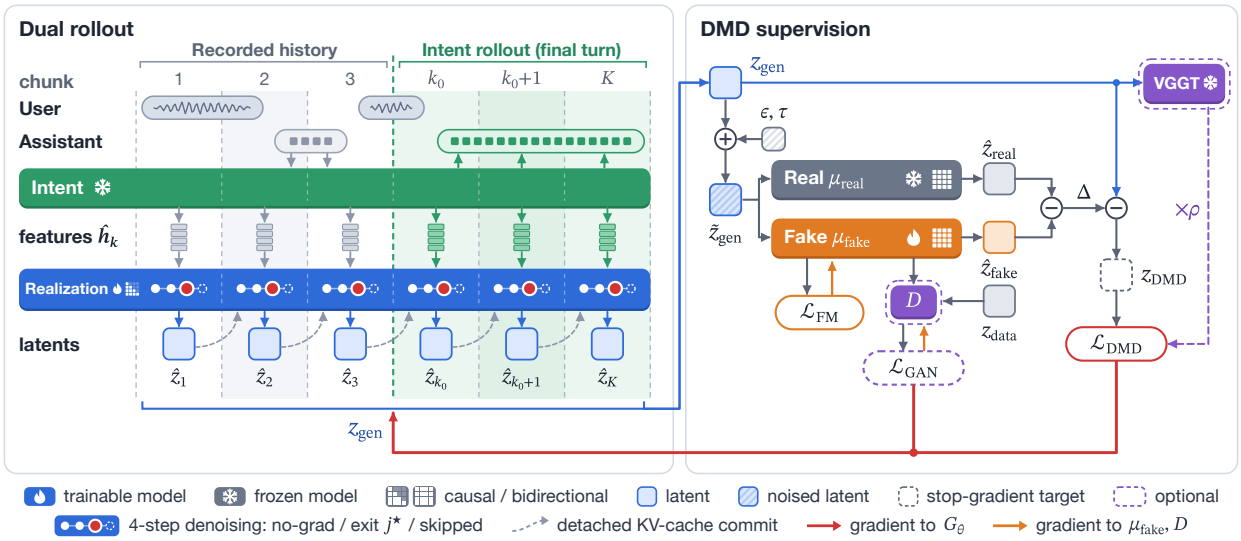


Figure 6: **DMD training with dual rollout.** The frozen Intent Module replays recorded assistant text and generates the final assistant turn from chunk k_0 onward. Its hidden states $\hat{h}_k$ condition the Realization Module G_θ, which generates the latent window z_{gen} chunk by chunk. The frozen real score model μ_{real} and the trainable fake score model μ_{fake} evaluate the corrupted latent window to construct the target z_{DMD}. Red arrows indicate gradients to G_θ. Orange arrows indicate gradients to μ_{fake} and D. Dashed outlines indicate optional components.

Here, C_k^ϕ contains recorded user inputs and earlier Intent Module outputs within the window, x_d^t embeds the global instructions, and $\widehat{h}_k$ are layer 50 states captured during the actual forward passes without an additional pass over generated text. With $N = 4$, $G_\theta^{(j)}$ performs j steps without gradient tracking followed by one differentiable step whose clean estimate becomes the chunk output. The exit $j^\star$ is shared across the trajectory. Each detached clean output is processed by a clean forward at $\tau = 1$ to update the KV cache. The assembled window $z_{\text{gen}} = (\widehat{z}_k)_k$ supplies the targets and losses in Eq. (10) and Eq. (12). The recorded turn boundary defines the training window without constraining the length or content of the generated response.

Auxiliary camera-rotation-aware weighting. DMD can introduce camera-rotation drift that distribution matching does not explicitly penalize. We therefore reweight the video DMD loss using rotations R_i estimated by frozen VGGT (Wang et al., 2025) from decoded rollout frames. Let α_i denote the geodesic angle between R_0 and R_i, and δ_i that between consecutive rotations. Sustained deviation and accumulated rotation define

$$E = \frac{1}{T_0}\int \left(\frac{[\alpha(s) - \alpha_{\text{dz}}]_+}{\alpha_{\text{s}}}\right)^2 ds + \left(\frac{1}{\alpha_{\text{p}}}\sum_i \delta_i \frac{g_{i-1} + g_i}{2}\right)^2, \qquad g_i = \text{clip}\left(\frac{\alpha_i - \alpha_{\text{dz}}}{\alpha_{\text{g}}}, 0, 1\right), \tag{14}$$
$$\rho = 1 + \lambda_{\text{rot}}\left(\max\left(\rho_{\max} e^{-E}, \rho_{\min}\right) - 1\right),$$

Here, s is playback time, α_{dz} defines a dead zone, and α_{s}, α_{g}, and α_{p} scale deviation, gating, and rotational path length, respectively. The constant T_0 normalizes duration, with weight bounds $0 < \rho_{\min} < 1 < \rho_{\max}$ and strength $\lambda_{\text{rot}} \in [0, 1]$. The detached weight ρ scales only the video term in Eq. (11). At $\lambda_{\text{rot}} = 1$, stable windows receive weights up to $\rho_{\max}$, while strongly rotating windows receive $\rho_{\min}$. The metric uses rotation alone, and opposite rotations do not cancel. This weighting rebalances DMD updates without propagating gradients through VGGT or the VAE. Intentional viewpoint changes and unreliable rotation estimates can also affect the weights.

5 Data Construction

Training **Duo v1** requires temporally aligned user inputs and assistant audiovisual behavior. We combine synthetic and recorded interactions into a unified conversation timeline, preserving natural timing and controlled events. This provides causal supervision for Intent and audiovisual targets for Realization, with additional routes for missing-view reconstruction and recorded-video extensions.

5.1 Synthetic Data

Pipeline overview. This route generates a two-person video call in which both participants are rendered together. Fig. 7 shows its two stages. **Stage I** is text only. Control cards are sampled, a LLM writes a structured screenplay under these cards, rule checks reject invalid screenplays, and a template converts each screenplay block into a text prompt (MiniMax, 2026). **Stage II** feeds these prompts to a t2v model, which generates one split-screen audio-video stream block by block. The left panel is always the user and the right panel the assistant. Generating both panels and the shared audio on a unified timeline preserves the timing of dialogue, reactions, overlaps, and interruptions.

Control cards. Without explicit constraints, LLM-generated conversations tend to repeat similar topics, relationships, and turn-taking patterns. To improve diversity, we use control cards that define attributes with predefined candidate values. For each screenplay, these values are sampled uniformly and included as explicit requirements in the generation prompt.

- duration: {30 s, 45 s, 60 s}
- language: {Chinese, English}
- persona: {none, relationship, occupation}, *e.g.*, relationship $\in$ {"parent-child", "colleagues", "doctor-patient", ...} (12 values), occupation $\in$ {"firefighter", "nurse", "chef", ...} (11 values, per participant)
- micro-actions: {off, on}, where "on" suggests 24 small reactions, *e.g.*, {"nod", "raise an eyebrow", "look away briefly", ...}
- interaction: {"everyday exchange", "interruption", "follow-up / debate", "proactive inquiry"}
- emotion: fixed to "follow the topic and the conversation"
- resolution: {1536 $\times$ 576, 1152 $\times$ 768}, used only by video synthesis

Each `interaction` value carries a written specification. For example, "interruption" requires one speaker to cut in before the other finishes, a brief overlap of voices, and the interrupted speaker yielding the turn.

Screenplay and t2v prompt synthesis. The sampled cards are inserted into a fixed template, and the LLM writes a synopsis followed by a JSON screenplay, shown in Fig. 8 (a). The screenplay describes each participant once (appearance, voice, environment, initial_pose) and then splits the call into blocks of 5-15 s at natural turn boundaries. Each block is an ordered list of events. An event pairs a description of expressions, gestures, and the listener's reaction with at most one speech entry, so nonverbal reactions need no invented dialogue. A rule-based validator checks the structure, block timing, language, and the absence of capture-device words such as "phone" in the narration, and returns errors to the LLM for up to 3 attempts.

The validated screenplay is converted into one t2v prompt per block by a template, without further LLM calls (Fig. 8(b)). Screenplay fields fill the slots verbatim. The shaded sentences are fixed for all samples and fall into three groups:

- **Reference roles.** Each input is assigned one job: the previous final frame is the starting state, the opening video only sets picture quality, and the opening audio only sets voices and room tone. Without these roles the model tends to replay the reference video or repeat its dialogue.
- **Picture content.** Each panel is the participant's own front camera, which rules out third-person, mirror, and behind-the-back views. Recording devices, UI elements, and subtitles, which otherwise appear frequently in generated calls, are excluded, and content may not cross the center divider, so the two crops stay clean.
- **Causal order.** A reaction may start only after its trigger has finished, so the assistant view never responds before the user acts.

Video synthesis. As shown in Fig. 7, Block 0 receives its prompt and a white layout image whose two panels are separated by a thin black divider. The layout image fixes only the split-screen geometry, and the model creates both participants and backgrounds from the prompt.

- **Previous final frame** (orange). The last frame of block $i-1$ is temporally aligned with the first target frame, preserving pose, expression, and framing across blocks.

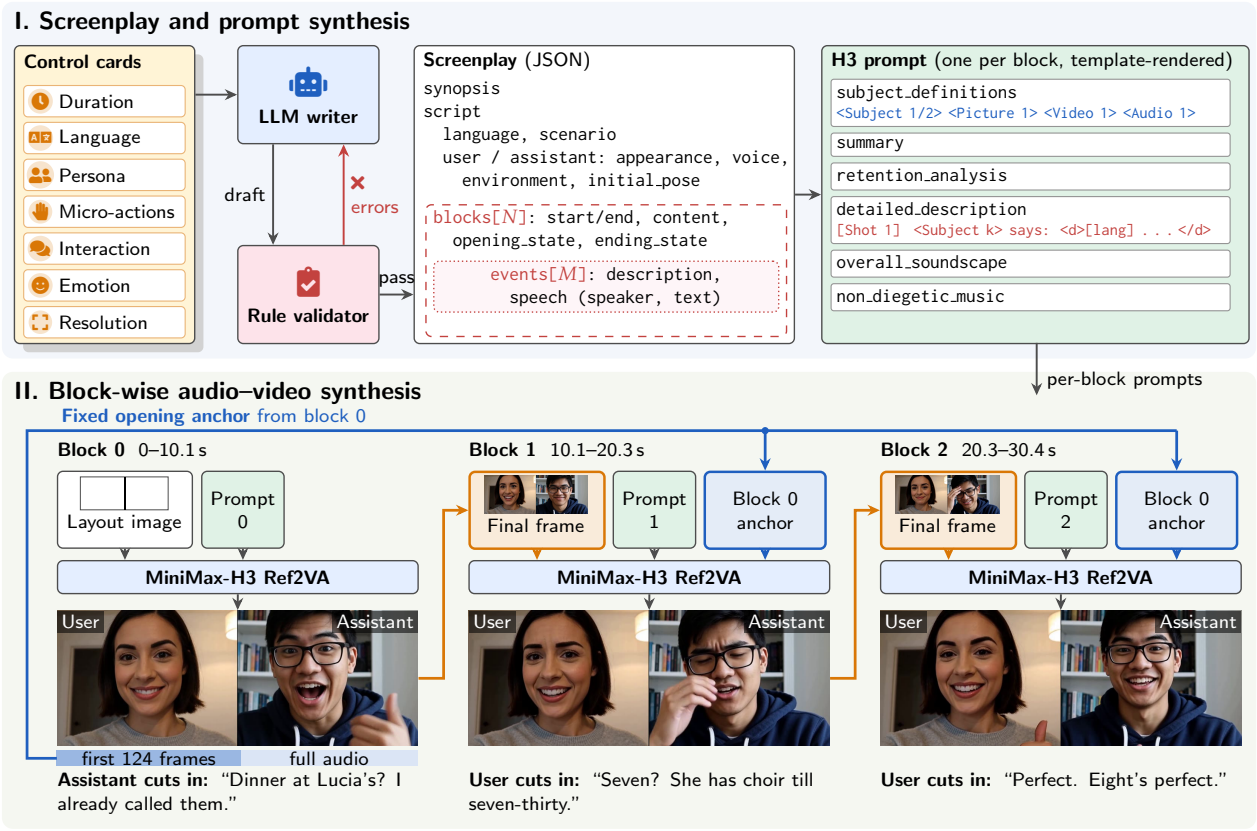


Figure 7: **Screenplay-driven dual-view generation.** Stage I turns sampled control cards into a validated structured screenplay and renders it, without another LLM call, into one t2v prompt per block. Stage II generates one split-screen audio-video block by block. Each later block starts from the previous block's final frame (orange) and is anchored to the same opening segment of block 0 (blue), so identity and voice references are never re-chained through earlier continuations. Frames and dialogue in this figure come from one generated 30 s sample.

- **Opening video** (blue). The first 124 frames of block 0 serve as a picture-quality reference for brightness, exposure, color sharpness, and noise.
- **Opening audio** (blue). The full block 0 audio anchors both voices and room tone.

We preserve generation conditions in their original representations, using VAE latents for the initial video and raw RGB for the final frame. The generated video are then concatenated and split into user and assistant views, both sharing the same synchronized audio track.

Error accumulation. The baseline data synthesis method conditions each block on the final frame of the preceding block, causing visual errors to accumulate over time. Our synthesis method mitigates this drift by using the opening video (block 0) as a fixed reference throughout generation. Fig. 9 compares the two synthesis strategies over a 60 s sequence of ten 6 s blocks. The baseline loses 34.2 luma levels by block 9, with increasingly exaggerated facial textures. Our method stabilizes at approximately 99 luma after a slight initial decline, maintaining consistent facial appearance through block 9.

5.2 Recorded Data

As shown in Fig. 10, we organize recorded data into six categories by source and interaction format: single-stream video, synchronous dual-stream video, asynchronous dual-stream video, live streams with audience comments, duplex audio recordings and open datasets. The following common recording standards apply across these categories. Recordings feature real speakers and their original voices, with speaker identities, transcripts, and temporal mappings retained and unrelated audio excluded. For video data, each delivered clip preserves a continuous, uncut half-body or full-body view with accurate audio–visual synchronization,

(a) Screenplay (JSON)

```
{"synopsis": 30 s call, a sister and her brother plan their
   mother's birthday ...
 "script": {
  "language": "English",
  "scenario": she calls about the plan, three interruptions
    ...,
  "user": {                                          left panel
   "appearance": woman in her twenties ...,
   "voice": clear young voice ...,
   "environment": handheld close-up, off-white wall,
   "initial_pose": standing, smiling at the lens},
  "assistant": { . . . },                            right panel
  "blocks": [ . . . ,
   {"block_index": 1,
    "start_seconds": 10, "end_seconds": 20,
    "content": he explains the booking, she cuts in,
    "opening_state": the user listens, the assistant leans
      forward,
    "events": [
     {"description": the assistant spreads his hands, the
        user's smile stops,
      "speech": {"speaker": "assistant",
       "text": "Yeah, Saturday at seven, as a
          surprise, so---"}},
     {"description": the user cuts in, voices overlap, the
        assistant stops,
      "speech": {"speaker": "user",
       "text": "Seven? She has choir till
          seven-thirty."}},
     {"description": the assistant touches his forehead,
      "speech": {"speaker": "assistant",
       "text": "Oh no. Right."}}],
    "ending_state": the user presses her lips, the assistant
      looks dismayed},
   . . . ]}}
```

(b) Prompt for a continuation block

subject_definitions:

<Subject 1> is the left-panel person, <Subject 2> the right-panel person, as shown in <Picture 1>.

[1] <Picture 1> is the immediately preceding block's final native RGB frame. It provides the intended first visual state of this new block.

<Video 1> is the first 124 frames of the opening block. It is a picture-quality reference only: it supplies the original brightness, exposure, color balance, sharpness, noise level and overall image quality, so that these do not drift across blocks.

<Audio 1> is the complete soundtrack of the opening block. It supplies the voice characteristics of the left-panel speaker and right-panel speaker, plus their room tone ...

summary:

[keyframe completion + image-quality reference + audio reference] Generate the next ⟨*duration*⟩ seconds of the same call ...

retention_analysis:

<Picture 1>: fully_preserved ... <Video 1> (image quality): reference ... <Audio 1>: reference ...

detailed_description:

[Shot 1] The shot begins from <Picture 1> ...

[2] This is a video of a remote call between two people, with one person generated in the left panel and the other in the right panel.

Each panel is exactly and only what that person's own front-facing camera captures: the person facing the lens, framed head-and-shoulders or head-and-upper-body.

The recording device is strictly off-screen and must never be visible: no phone, tablet, laptop, camera, tripod, stand, mount, screen or selfie stick anywhere in the frame, in anyone's hand, on any surface, or as a reflection.

No interface, UI, subtitles, speech bubbles or overlaid text.

All generated content must remain within its own panel and must not cross or cover the center divider.

[3] Every response, action or visual change happens only after the line or event that triggers it has finished, in the order described below, never earlier.

<Subject 1> left person: ⟨*appearance*⟩ voice: ⟨*voice*⟩ environment: ⟨*environment*⟩

<Subject 2> right person: ...

Opening state: ⟨*opening_state*⟩

⟨*description*⟩

<Subject 2>, the person in the right panel, says: <d>[English] ⟨*text*⟩</d>

... *(one pair per event)*

Ending state: ⟨*ending_state*⟩

overall_soundscape:

Use <Audio 1> as a reference for room tone, without copying its waveform or dialogue.

non_diegetic_music:

N/A

Figure 8: **Screenplay and t2v prompt.** (a) The screenplay written by the LLM for the sample in Fig. 7. (b) The t2v prompt rendered from it for a continuation block. Shaded sentences are fixed text shared by all samples, grouped as [1]-[3] in the main text. ⟨·⟩ marks screenplay fields copied into the prompt, and the remaining lines are abridged template text.

at least 720p effective resolution per view, and no visual overlays. Dual-stream recordings contain at least one complete conversational exchange.

Single-stream video. Fig. 10 (b) illustrates single-stream recordings, such as solo podcasts and single-person presentations. The clips range from 10 to 90 seconds, preserving natural pauses.

Synchronous dual-stream video. Synchronous dual-stream video, illustrated in Fig. 10 (c), keeps both participants visible at the same time and preserves listening behavior, overlapping speech, and reactions.

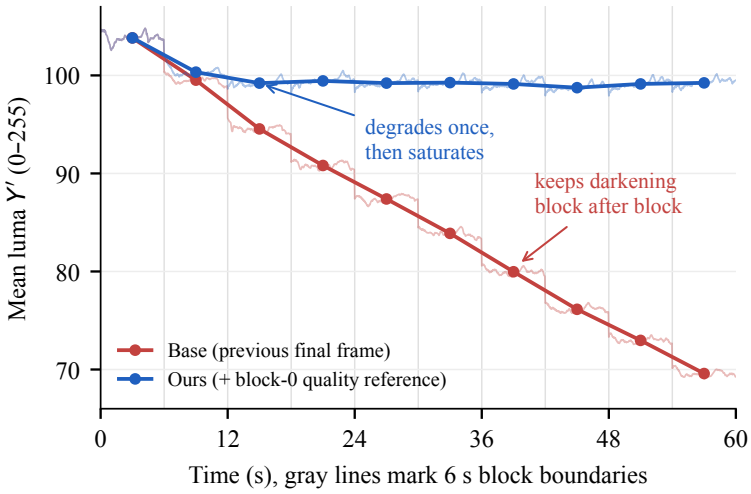
Asynchronous dual-stream video. Asynchronous dual-stream video shows participants in separate clips that alternate with the speaking turns, as in the interview footage shown in Fig. 10 (d).

Live streams with audience comments. We align timestamped audience questions with presenter responses and reconstruct user videos, converting live streams into synchronized dual-stream conversations.

Duplex audio recordings. This category consists of full-duplex audio recordings of two-person dialogues, which drive the further video synthesis, as illustrated in Fig. 10 (f).

Open datasets. We collected single-stream talking videos from open-source datasets. As depicted in Fig. 10 (g), we filtered the material according to the common recording standards (e.g., resolution require-

Mean luma Y'	Base	Ours
Block 0	103.8	103.8
Block 1	99.5	100.3
Block 2	94.5	99.2
Block 3	90.8	99.4
Block 4	87.4	99.2
Block 5	83.9	99.3
Block 6	80.0	99.1
Block 7	76.1	98.7
Block 8	73.0	99.1
Block 9	69.6	**99.2**
Block 9 − Block 0	−34.2	−4.6
Block 9 − Block 1	−29.9	−1.1
Slope (Y'/s)	−0.63	−0.05



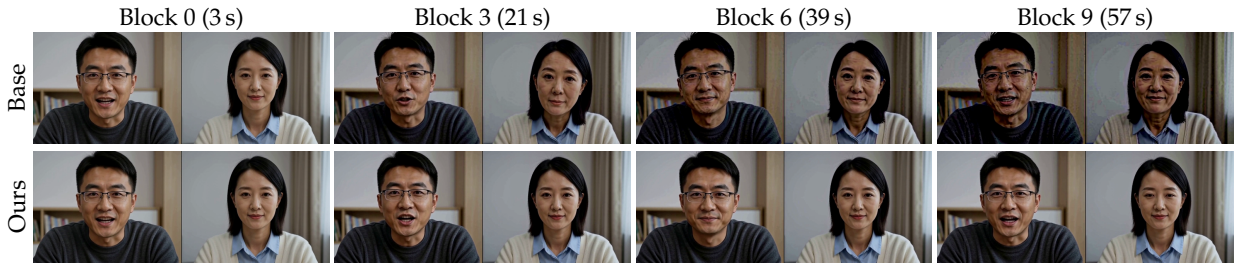


Figure 9: **Error accumulation over a 60 s, ten-block rollout.** Top: mean luma Y' (0–255) of the full split-screen frame per block (table) and per frame (plot). Bottom: the split-screen frame at the middle of blocks 0, 3, 6, and 9. Base conditions each block only on the previous final frame, and Ours adds the fixed block-0 picture-quality reference. Both runs share the same block 0 and screenplay.

ments) and then applied face detection, video understanding, voice activity detection (VAD), automatic speech recognition (ASR), and language identification to guide further filtering and annotation.

5.3 Data Processing

5.3.1 Synthetic Data Quality Control

Each newly generated video segment undergoes four automatic quality checks.

First-frame consistency. We compare the first frame with the conditioning image, requiring an SSIM of at least 0.90 and a perceptual-hash Hamming distance of at most 4. Conditioning uses the character image initially, then the preceding segment's processed final frame.

Unexpected audio. We compare the generated audio with the source audio using loudness normalization, waveform residuals, and VAD, rejecting segments with unexplained audio activity lasting at least 0.40 s.

Camera push-in. We track face scale at 4 FPS, using the first valid detection as the baseline. Rejection thresholds for sustained and abrupt enlargement are a smoothed scale of 1.30 times baseline for 0.75 s and a 1.22-fold jump within 0.50 s, respectively.

Persistent large-area visual changes. We sample frames at 4 FPS and measure LAB differences, changed-pixel proportions, and color/brightness histogram distances. Using these metrics, we reject abrupt changes that remain above thresholds relative to the pre-change frame, or cumulative drift from the initial frame exceeding both appearance and histogram thresholds.

5.3.2 Live-Stream Data Processing

Collection and initial screening. We collect archived live streams and deduplicate source records. Sampled frames are used to check the presence of real participants and visual quality, retaining only suitable material.

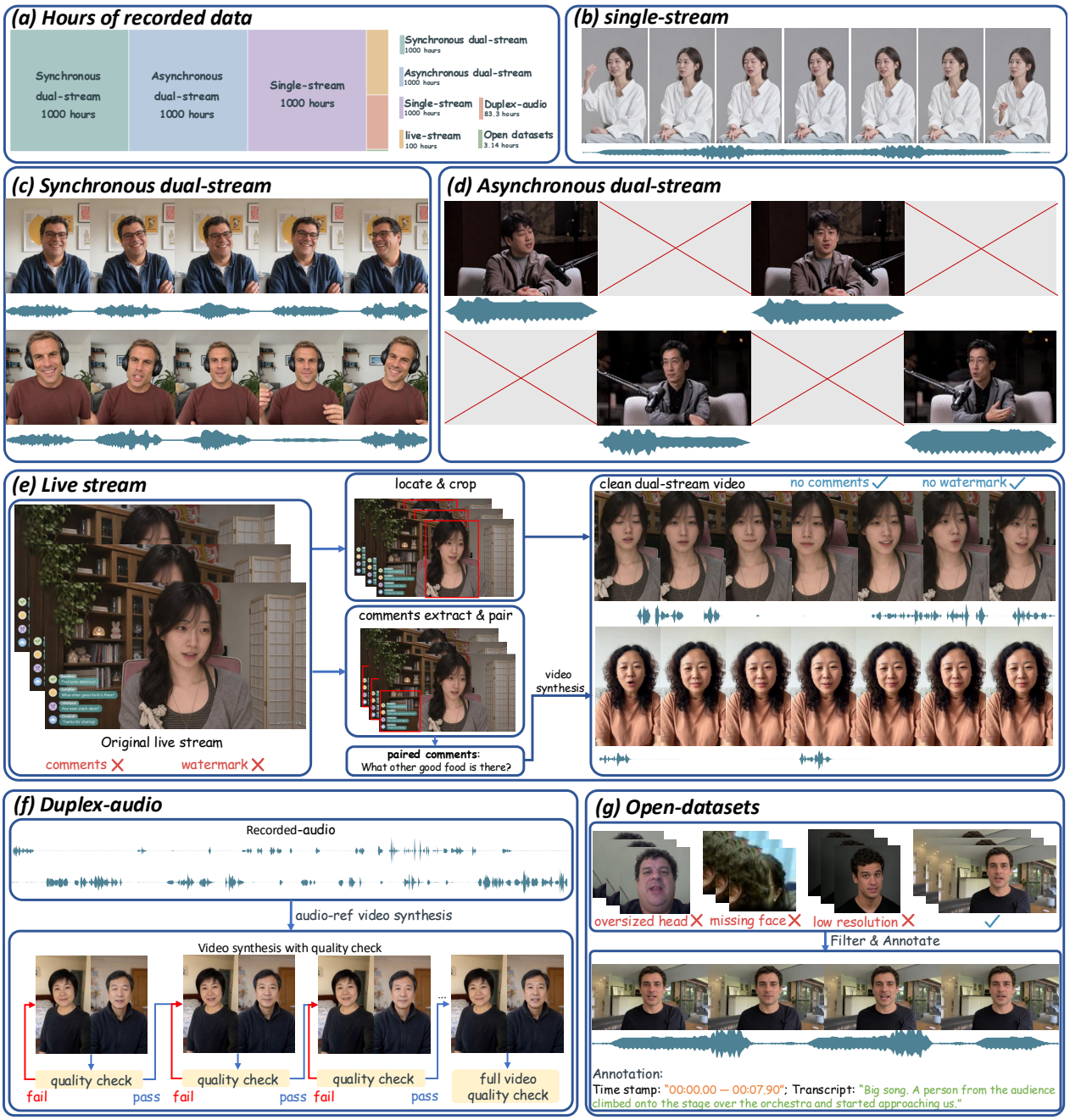


Figure 10: **Recorded data sources, interaction formats** The six categories describe the recorded sources, with selected categories requiring further processing or synthesis to construct training examples.

Annotation and filtering. We produce transcripts with word- or character-level timestamps by Qwen3-ASR-1.7B and Qwen3-ForcedAligner-0.6B (Shi et al., 2026). And we also use Silero VAD (Silero Team, 2024) to identify speech and silence intervals and Qwen2.5-Omni-7B (Xu et al., 2025a) to review audio types. Speaker diarization and speaker-count checks combine FSMN-VAD (Gao et al., 2023) for speech detection, Paraformer (Gao et al., 2022)for transcription, and CAM++ (Wang et al., 2023) for speaker embeddings followed by clustering. These checks exclude windows containing singing, whispered or breathy speech, or multiple speakers, producing timestamped candidate material for dialogue construction.

Dialogue construction and video preparation. To obtain the content and timing needed for user-side video synthesis, we match audience comments extracted from real chat logs to the presenter's responses.

When such matching is not possible, we will generate counterpart utterances and schedule their interjections based on the presenter's original speech and available silence. After reviewing conversational coherence, completeness, and audio-visual correspondence, we extract continuous assistant-side video segments and crop them around the presenter to remove on-screen comments and watermarks. We then synthesize the corresponding user-side videos and pair the two views on a shared timeline. After a final quality check, we deliver clean dual-stream videos together with their transcripts, dialogue timing and source metadata.

6 Streaming Pipeline and Acceleration

In this section, we detail our streaming inference design. The system employs an orchestrated scheduling strategy to synchronize the intent module, the realization module and the VAE decoding, ensuring low-latency performance for interactive tasks.

6.1 Orchestrated Streaming Pipeline

The front-end device continuously captures user audio and video and forwards them for chunk-wise processing. As shown in Fig. 11, each chunk passes through input acquisition, intent inference, joint latent generation, and streaming decoding, while input capture continues during response generation. The sequential stages of the pipeline can be summarized as:

- **Interaction sampling.** At 24 frames per second, each sampling window contains 17 video frames and the corresponding user audio, matching the video VAE grouping described in Sec. 3.3. The window duration is $t_{\text{sample}} = 17/24\,\text{s} \approx 708\,\text{ms}$.
- **Intent inference.** The Intent Module processes the available context, generates a participation and text output, and provides hidden states to the Realization Module. Depending on the input and context length, inference takes $t_{\text{intent}} = 90\text{–}160\,\text{ms}$.
- **Joint latent generation.** Conditioned on the intent representation and media context, the Realization Module jointly generates audio and video latents. For video generation at 768×576 resolution, this stage takes $t_{\text{gen}} = 200\text{-}450\,\text{ms}$, depending on the context length.
- **Streaming decoding.** The generated audio and video latents are decoded incrementally for playback. Decoding a 4-frame group takes approximately $t_{\text{decode}} = 30\,\text{ms}$.
- **Other overhead.** Data movement, transmission to the front-end device, and display processing contribute approximately $t_{\text{overhead}} = 40\,\text{ms}$.

The reported intent inference, latent generation, and overhead terms sum to

$$t_{\text{intent}} + t_{\text{gen}} + t_{\text{overhead}} \approx 330\text{–}650\,\text{ms}. \tag{15}$$

The delay to the first visible output additionally includes the initial decoding time and any remaining wait for the sampling window to close. An utterance or action ending within a sampling window can incur an additional sampling delay of up to approximately 708 ms. During playback, we post-process the generated frames to maintain visual stability and smooth motion.

6.2 Causal Latent Decoding

The original VAE in (MiniMax, 2026) decodes 5 latent frames into 17 video frames and requires 2 latent frames of lookahead. This dependency introduces additional waiting at chunk boundaries during streaming playback. We therefore adapt the decoding procedure to process available video latents incrementally.

After joint generation of a response chunk, its video latents are decoded one at a time. Each latent produces four video frames, followed by temporal alignment to match the original playback schedule and preserve the 17-frame chunk duration. This procedure allows playback to begin from the available chunk without waiting for additional latents from the next chunk.

At 24 frames per second, the shortest playback interval corresponds to the first latent in each group, which represents one video frame after temporal alignment. This interval is $1/24\,\text{s} \approx 41.7\,\text{ms}$. The measured per-latent decoding time is approximately 30 ms, below this interval. The decoder can therefore keep pace with playback provided that generated latents are available on schedule.

6.3 Inference Acceleration

The runtime coordinates memory management, stage scheduling, and GPU placement to sustain continuous audiovisual generation. Each input chunk covers $17/24\,\text{s}$ of captured audio and video. Sustained playback

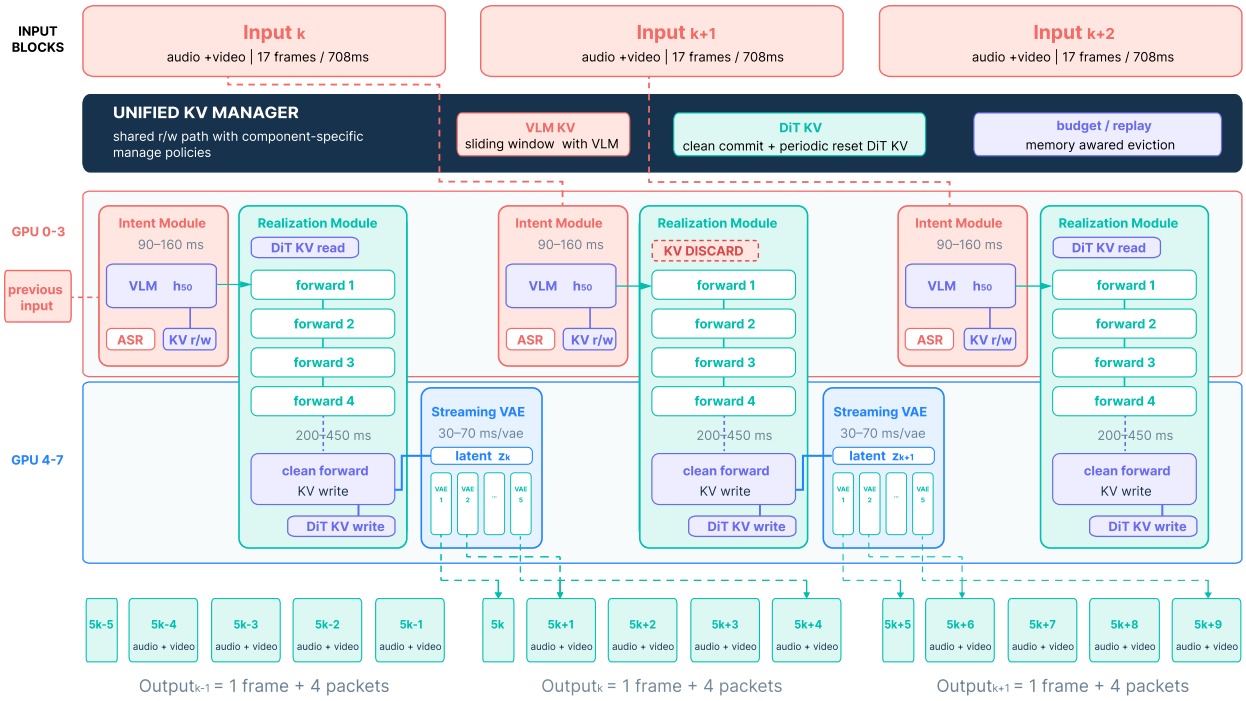


Figure 11: **Streaming inference on eight GPUs.** Time advances from left to right. The Intent Module runs its VLM with four-way tensor parallelism (TP4) on GPU 0-3, alongside auxiliary ASR. The Realization Module executes the DiT on GPU 0-7 using tensor or sequence parallelism (TP/SP), while the streaming VAE uses 4-way spatial tiling on GPU 4-7. The first visible execution of the Intent Module processes input k, which is omitted from the figure. The second execution processes input $k+1$. Hidden states h_k from the Intent Module condition the Realization Module, which performs 4 denoising steps followed by a clean forward that commits the KV state. The unified KV manager uses a sliding window for Intent Module KV and periodic resets for Realization Module KV. Chunk $k+1$ skips history reads but retains the clean write. Decoding overlaps intent inference on separate GPUs before generation resumes. KV boxes mark manager operations.

requires the pipeline to produce output chunks at least as quickly as they are consumed, while keeping memory usage bounded over long interactions. As shown in Fig. 11, the system runs on a single server with eight GPUs. For each response chunk, Intent Module inference precedes joint audiovisual latent generation by the DiT. The remaining VAE decoding steps for one chunk overlap with Intent Module inference for the following chunk, reducing the interval between successive outputs.

Unified KV Management A shared KV cache manager tracks cache ownership and device placement, with sliding-window eviction for the Intent Module and periodic history resets for the Realization Module. For each chunk, four denoising forwards read committed historical KV without committing intermediate noisy states. An additional *clean forward* processes the generated latents z_k^a and z_k^v and writes their KV states for subsequent chunks.

At reset boundaries, the manager discards historical media KV before denoising while retaining reference and shared conditioning. The clean forward then establishes a new history, as illustrated by chunk $k+1$ in Fig. 11. The reset interval controls retained context, memory use, and context-dependent latency. After a restart, the manager reconstructs the active media cache by replaying completed latents through clean forwards in their original order, following the same reset boundaries and retention policy.

Chunk-Level Overlap and Deadline Scheduling Input capture proceeds independently of computation. As shown in Fig. 11, the Realization Module uses all 8 GPUs for denoising and the clean forward. Between generation phases, GPUs 0-3 run Intent Module inference for chunk $k+1$, while GPUs 4-7 decode chunk k. Generation resumes once both groups finish. After temporal alignment, each 17-frame chunk is released incrementally as one 1-frame anchor followed by four 4-frame packets, each synchronized with its audio.

Let $t_{\text{gen},k}$ include four denoising forwards and the clean forward, $t^{\text{full}}_{\text{decode},k}$ denote complete chunk decoding, $t_{\text{intent},k+1}$ denote the next intent inference, and $t_{\text{switch},k}$ denote additional transition overhead. When the next input context is available and decoding starts together with intent inference, the cycle time is

$$T_{\text{cycle},k} = t_{\text{gen},k} + \max\left\{t_{\text{intent},k+1}, t^{\text{full}}_{\text{decode},k}\right\} + t_{\text{switch},k}. \tag{16}$$

The target is $T_{\text{cycle},k} \leq 17/24\,\text{s}$, with margin for stage-time variation. Complete decoding enters the throughput budget, whereas only first-packet decoding enters the initial response path. The scheduler reserves decoding slots, bounds output lead over playback, and admits work using KV windows, available memory, and estimated stage durations.

Device-Aware Parallel Execution The Intent Module uses four-way tensor parallelism (TP), while the Realization Module selects between TP and sequence parallelism (SP). In our SP configuration, each rank holds a complete weight replica and processes part of the token sequence. TP partitions weights and tensor computation. For chunk k, the runtime selects the fastest feasible configuration:

$$\begin{aligned} p_k^\star &= \arg\min_{p\in\{\text{SP},\text{TP}\}} T_k(p), \\ \text{s.t.} \quad & M_{k,g}(p) \leq B_{k,g}, \quad g = 0,\ldots,7. \end{aligned} \tag{17}$$

Here, $T_k(p)$ denotes profiled runtime, $M_{k,g}(p)$ peak memory for weights, retained KV, communication buffers, and temporaries, and $B_{k,g}$ memory remaining after other resident stages. We retain weights when memory permits and prefetch nonresident weights during slack, accounting for transfer and synchronization costs. ASR, input transfers, and packet preparation use pipeline gaps as data dependencies allow.

7 Evaluation and Showcase

7.1 Human Evaluation

Evaluation protocol We evaluated the model on 30 real-time duplex interaction cases, each pairing a user's audio-visual input with the model's generated audio-visual reply, replayed side by side on a shared timeline. Each reply is judged the way an end user would experience it, at normal playback speed with sound, against six binary criteria: content responsiveness, language adherence, speech intelligibility, visual identity stability, audio-visual synchronization, and audio-visual emotional consistency. Every criterion has written failure conditions, and no partial credit is given.

Results Of 180 checks (30 cases $\times$ 6 criteria), 168 passed (93.3%). The clearest strength is generation fidelity: the model passed all 90 checks on language adherence, speech intelligibility, and visual identity stability. Every reply came in the expected language as complete, intelligible speech, and no case showed a visible identity swap, facial or body malformation, or unexplained change in appearance. Cross-modal coherence is nearly as strong. Vocal emotion was consistent with facial expression and body language in 96.7% of cases, and lip movement stayed aligned with speech in 90.0%.

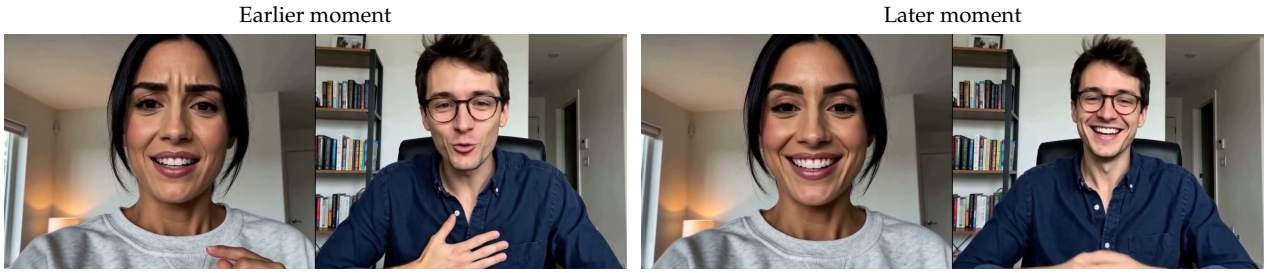
Where headroom remains Content responsiveness, which asks whether a reply substantively addresses what the user has just said and shown, reached 73.3% (22/30). It is the most demanding criterion in the protocol, since a duplex model must settle on a reply while speech and video are still streaming in. The misses fall mainly into two patterns. The first is the long tail of conversation, such as jokes and odd remarks, where a fitting reply requires reading intent beyond the literal words, and doing so in real time. The second is intent that only becomes clear when speech and video are read together. In one case, a user sitting indoors said they had just unlocked a shared bike when it started to rain. Given the indoor setting, this reads as a complaint, yet the model treated it as a request for advice and suggested holding off on riding. Both patterns come down to inferring what the user means from everything they say and show, at conversational speed. This is one of the hardest problems in duplex interaction and the central focus of our next iteration, which builds on a foundation that already passed every language, speech, and identity check.

7.2 Showcase

Fig. 12 organizes selected examples: plan revision, visual guidance, collaborative design, social support, and interactive customization. These categories connect language to a change in a shared plan, an observable action, a joint solution, or an affective response. The first four categories use script-conditioned training data, the customization example comes from the user demonstration. Fig. 13 extends the coverage to negotiation, decision support, explanation, and task coordination. Fig. 14 adds collaborative creation, coaching, shared experience, turn negotiation, and supportive responses.

8 Conclusion

We introduced the `Reciprocal World Model` and its initial implementation, **`Duo v1`**, which couples a multimodal Intent Module with a causal audiovisual Realization Module. This report describes the architecture, training procedure, and runtime scheduling used to generate responses while continuing to receive user input. **`Duo v1`** remains preliminary and currently supports basic audiovisual conversation. The examples illustrate this functionality, while response timing, consistency over longer sessions, and recovery from misunderstandings still require broader evaluation. Development is ongoing, and an updated version will follow soon.



(a) Plan revision. Resolve a schedule conflict and agree on a new plan.



(b) Visual guidance. Inspect a plant and demonstrate watering advice.



(c) Collaborative design. Compare samples and revise a surface treatment.



(d) Social support. Turn an awkward apology into a reassuring exchange.



(e) Interactive customization. Reflect a visible reference in a garment graphic, then revise it.

Figure 12: Five functions of user–assistant interaction. Examples connect dialogue to plans, visible actions, shared solutions, social responses, and interactive customization. Panels (a)-(d) show offline, script-conditioned paired-video data. Panel (e) shows successive moments from a provided project demo.



(a) Plan negotiation



(b) Plan revision



(c) Decision support



(d) Reasoned disagreement



(e) Visual explanation



(f) Task coordination

Figure 13: **Decisions and coordination.** Six additional script-conditioned paired-video examples illustrate how the user and assistant question a proposal, revise a plan, reason through constraints, explain visible evidence, and coordinate a shared task.



(a) Collaborative creation



(b) Adaptive coaching



(c) Shared experience



(d) Mutual care



(e) Turn negotiation



(f) Reassurance





(g) Emotional support

Figure 14: **Grounded and social responses.** Seven additional script-conditioned conversations connect visible context and dialogue to creative, instructional, and supportive behavior. Panels (a)-(f) show different conversations. Panel (g) shows two moments from one conversation.

References

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. doi: 10.48550/arXiv.2511.21631. URL https://arxiv.org/abs/2511.21631.

Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024. URL https://openai.com/research/video-generation-models-as-world-simulators.

Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024. doi: 10.48550/arXiv.2402.15391. URL https://arxiv.org/abs/2402.15391.

Boyuan Chen, Diego Marti Monso, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. In *Advances in Neural Information Processing Systems*, 2024. doi: 10.48550/arXiv.2407.01392. URL https://arxiv.org/abs/2407.01392.

Qiaoling Chen, Jiaxiang Cheng, Xueqing Deng, Tiankai Hang, Ji Hou, Jiaqi Li, Yifan Liu, Tao Lu, Junshu Tang, Yu Tang, et al. Hy-nomas: Towards free-form interactive world generation. *arXiv preprint arXiv:2610.36729*, 2026.

Ethan Chern, Zhulin Hu, Bohao Tang, et al. LiveTalk: Real-Time Multimodal Interactive Video Diffusion via Improved On-Policy Distillation. *arXiv preprint arXiv:2512.23576*, 2025. doi: 10.48550/arXiv.2512.23576. URL https://arxiv.org/abs/2512.23576.

Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: A speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024. doi: 10.48550/arXiv.2410.00037. URL https://arxiv.org/abs/2410.00037.

Zelin Gao, Qiuyu Wang, Jiapeng Zhu, Jingye Chen, Zichen Liu, Qingyan Bai, Jiahao Wang, Yufeng Yuan, Hanlin Wang, Yichong Lu, Ka Leong Cheng, Haojie Zhang, Jian Gao, Tianrui Feng, Yuzheng Liu, Yao Yao, Yinghao Xu, Xing Zhu, Yujun Shen, and Hao Ouyang. Infinite worlds with versatile interactions. *arXiv preprint arXiv:2607.07534*, 2026.

Zhifu Gao, Shiliang Zhang, Ian McLoughlin, and Zhijie Yan. Paraformer: Fast and accurate parallel transformer for non-autoregressive end-to-end speech recognition. In *Interspeech 2022*, pp. 2063–2067, 2022. doi: 10.21437/Interspeech.2022-9996. URL https://www.isca-archive.org/interspeech_2022/gao22b_interspeech.html.

Zhifu Gao, Zerui Li, Jiaming Wang, Haoneng Luo, Xian Shi, Mengzhe Chen, Yabin Li, Lingyun Zuo, Zhihao Du, and Shiliang Zhang. Funasr: A fundamental end-to-end speech recognition toolkit. In *Interspeech 2023*, pp. 1593–1597, 2023. doi: 10.21437/Interspeech.2023-1428. URL https://www.isca-archive.org/interspeech_2023/gao23g_interspeech.html.

Google DeepMind. Genie 2: A large-scale foundation world model. Official research release, 2024. URL https://deepmind.google/blog/genie-2-a-large-scale-foundation-world-model/. Published December 4, 2024; accessed October 30, 2026.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. doi: 10.48550/arXiv.1803.10122. URL https://arxiv.org/abs/1803.10122.

Yoav HaCohen, Benny Brazowski, Nisan Chiprut, et al. LTX-2: Efficient Joint Audio-Visual Foundation Model. *arXiv preprint arXiv:2601.03233*, 2026. doi: 10.48550/arXiv.2601.03233. URL https://arxiv.org/abs/2601.03233.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025. doi: 10.1038/s41586-025-08744-2. URL https://www.nature.com/articles/s41586-025-08744-2.

Anthony Hu, Lloyd Russell, Hudson Yeo, et al. GAIA-1: A Generative World Model for Autonomous Driving. *arXiv preprint arXiv:2309.17080*, 2023. doi: 10.48550/arXiv.2309.17080. URL https://arxiv.org/abs/2309.17080.

Junchao Huang, Guian Fang, Shengju Qian, et al. SolarWM: Open data and scalable training for long-horizon video world models. *arXiv preprint arXiv:2609.02886*, 2026a. doi: 10.48550/arXiv.2609.02886. URL https://arxiv.org/abs/2609.02886.

Lianghua Huang, Zhi-Fan Wu, Wei Wang, et al. Wan-Streamer v0.1: End-to-end real-time interactive foundation models. *arXiv preprint arXiv:2606.25041*, 2026b. doi: 10.48550/arXiv.2606.25041. URL https://arxiv.org/abs/2606.25041.

Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. In *Advances in Neural Information Processing Systems*, 2025. doi: 10.48550/arXiv.2506.08009. URL https://arxiv.org/abs/2506.08009.

Gaojie Lin, Jianwen Jiang, Jiaqi Yang, et al. OmniHuman-1: Rethinking the Scaling-Up of One-Stage Conditioned Human Animation Models. *arXiv preprint arXiv:2502.01061*, 2025. doi: 10.48550/arXiv.2502.01061. URL https://arxiv.org/abs/2502.01061.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.02747. URL https://arxiv.org/abs/2210.02747.

Chetwin Low, Weimin Wang, and Calder Katyal. Ovi: Twin Backbone Cross-Modal Fusion for Audio-Video Generation. *arXiv preprint arXiv:2510.01284*, 2025. doi: 10.48550/arXiv.2510.01284. URL https://arxiv.org/abs/2510.01284.

MiniMax. MiniMax H3: Official model card. Hugging Face model card, 2026. URL https://huggingface.co/MiniMaxAI/MiniMax-H3. Accessed October 30, 2026; year denotes the access year of the undated model card.

NVIDIA. Cosmos World Foundation Model Platform for Physical AI. *arXiv preprint arXiv:2501.03575*, 2025. doi: 10.48550/arXiv.2501.03575. URL https://arxiv.org/abs/2501.03575.

Jack Parker-Holder and Shlomi Fruchter. Genie 3: A new frontier for world models. Google DeepMind official model release, 2025. URL https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/. Published August 5, 2025; accessed October 30, 2026.

Robbyant Team. Advancing open-source world models. *arXiv preprint arXiv:2601.20540*, 2026. doi: 10.48550/arXiv.2601.20540. URL https://arxiv.org/abs/2601.20540.

Xian Shi, Xiong Wang, Zhifang Guo, et al. Qwen3-ASR Technical Report. *arXiv preprint arXiv:2601.21337*, 2026. doi: 10.48550/arXiv.2601.21337. URL https://arxiv.org/abs/2601.21337.

SII-OpenMOSS Team. MOVA: Towards Scalable and Synchronized Video-Audio Generation. *arXiv preprint arXiv:2602.08794*, 2026. doi: 10.48550/arXiv.2602.08794. URL https://arxiv.org/abs/2602.08794.

Silero Team. Silero vad: Pre-trained enterprise-grade voice activity detector (VAD), number detector and language classifier. GitHub software repository, 2024. URL https://github.com/snakers4/silero-vad. Software citation follows the official CITATION.cff; accessed November 7, 2026.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 32211–32252, 2023. doi: 10.48550/arXiv.2303.01469. URL https://proceedings.mlr.press/v202/song23a.html.

Yaofeng Su, Yuming Li, Zeyue Xue, et al. OmniForcing: Unleashing Real-time Joint Audio-Visual Generation. *arXiv preprint arXiv:2603.11647*, 2026. doi: 10.48550/arXiv.2603.11647. URL https://arxiv.org/abs/2603.11647.

Linrui Tian, Qi Wang, Bang Zhang, and Liefeng Bo. EMO: Emote Portrait Alive – Generating Expressive Portrait Videos with Audio2Video Diffusion Model under Weak Conditions. *arXiv preprint arXiv:2402.17485*, 2024. doi: 10.48550/arXiv.2402.17485. URL https://arxiv.org/abs/2402.17485.

Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines, 2025. URL https://arxiv.org/abs/2408.14837.

Hui Wang, Siqi Zheng, Yafeng Chen, Luyao Cheng, and Qian Chen. Cam++: A fast and efficient network for speaker verification using context-aware masking. In *Interspeech 2023*, pp. 5301–5305, 2023. doi: 10.21437/Interspeech.2023-1513. URL https://www.isca-archive.org/interspeech_2023/wang23ha_interspeech.html.

Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. doi: 10.48550/arXiv.2503.11651. URL https://arxiv.org/abs/2503.11651.

Siyin Wang, Wenyi Yu, Terumi Chiba, et al. Full-Duplex Interaction with Speech, Expressions and Gestures. *arXiv preprint arXiv:2610.21452*, 2026. doi: 10.48550/arXiv.2610.21452. URL https://arxiv.org/abs/2610.21452.

Jin Xu, Zhifang Guo, Jinzheng He, et al. Qwen2.5-Omni Technical Report. *arXiv preprint arXiv:2503.20215*, 2025a. doi: 10.48550/arXiv.2503.20215. URL https://arxiv.org/abs/2503.20215.

Jin Xu, Zhifang Guo, Hangrui Hu, et al. Qwen3-Omni Technical Report. *arXiv preprint arXiv:2509.17765*, 2025b. doi: 10.48550/arXiv.2509.17765. URL https://arxiv.org/abs/2509.17765.

Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Frédo Durand, and William T. Freeman. Improved distribution matching distillation for fast image synthesis. In *Advances in Neural Information Processing Systems*, 2024a. doi: 10.48550/arXiv.2405.14867. URL https://arxiv.org/abs/2405.14867.

Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024b. doi: 10.48550/arXiv.2311.18828. URL https://arxiv.org/abs/2311.18828.

Tianwei Yin, Qiang Zhang, Richard Zhang, William T. Freeman, Frédo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast autoregressive video diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. doi: 10.48550/arXiv.2412.07772. URL https://openaccess.thecvf.com/content/CVPR2025/papers/Yin_From_Slow_Bidirectional_to_Fast_Autoregressive_Video_Diffusion_Models_CVPR_2025_paper.pdf.

Min Zhao, Hongzhou Zhu, Kaiwen Zheng, et al. Causal Forcing++: Scalable Few-Step Autoregressive Diffusion Distillation for Real-Time Interactive Video Generation. *arXiv preprint arXiv:2605.15141*, 2026. doi: 10.48550/arXiv.2605.15141. URL https://arxiv.org/abs/2605.15141.

Hongzhou Zhu, Min Zhao, Guande He, et al. Causal Forcing: Autoregressive Diffusion Distillation Done Right for High-Quality Real-Time Interactive Video Generation. *arXiv preprint arXiv:2602.02214*, 2026a. doi: 10.48550/arXiv.2602.02214. URL https://arxiv.org/abs/2602.02214.

Lunjie Zhu, Xingtong Ge, Fangyu Lin, et al. Omni-LiveAvatar: Minute-Level Real-Time Streaming Joint Audio-Video Avatar Generation. *arXiv preprint arXiv:2608.13602*, 2026b. doi: 10.48550/arXiv.2608.13602. URL https://arxiv.org/abs/2608.13602.